

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

старший преподаватель
должность, уч. степень, звание

подпись, дата

Д. О. Шевяков
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №7

Создание и настройка системы сбора
данных на базе платформы интернета
вещей

по курсу: Интернет вещей

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г.В. Буренков
инициалы, фамилия

Санкт-Петербург 2026

СОДЕРЖАНИЕ

1 Цель работы	2
2 Задание	3
3 Ход выполнения задания.....	4
4 Вывод.....	7
ПРИЛОЖЕНИЕ А	8

1 Цель работы

Целью лабораторной работы является получение практических навыков организации долгосрочного хранения данных системы «умный дом» в документоориентированной базе данных MongoDB.

2 Задание

В соответствии с методическими указаниями для лабораторной работы №7 требуется реализовать систему логирования данных с использованием не менее двух методов записи в разные коллекции базы данных.

3 Ход выполнения задания

В ходе выполнения лабораторной работы сохранены функции мониторинга и управления устройствами, реализованные в предыдущих работах, и добавлен модуль журналирования данных `IoTLogger`. Подключение выполняется к MongoDB по адресу `mongodb://127.0.0.1:27017` (либо через переменную `MONGODB\_URI`) с выбором базы данных `iot\_logger\_db`. Сервер приложения запускается на порту 5007 и продолжает обслуживать сайт даже при недоступности базы данных. На рисунке 1 изображена страница приложения ЛР7 с мониторингом устройств..

ЛР7 — журнал в MongoDB

Порт по умолчанию 5007. Нужен MongoDB (`mongodb://127.0.0.1:27017` или `MONGODB_URI`). Коллекции `Temperature` и `Heater` .

Мониторинг

Температура: 10.7 °C

Свет: ВКЛ, яркость 1%

Замок: ОТКРЫТ

Обогреватель: ВКЛ (порог 25 °C)

Управление (с валидацией на сервере)

Температура

Примеры: `23.5` или `23,5`

```
{"ok":true,"state":{"id":"sensor-1","name":"Температурный датчик","value":23,"unit":"°C"},"heater":{"id":"heater-1","name":"Обогреватель","heaterPower":"On","switchOnBelowC":25}}
```

Свет

`enabled` — слово: on, off, true, false, да, нет ... Яркость — только цифры 0–100.

Замок

Строка из словаря: `закрыт`, `открыт`, `true`, `false`, ...

Рисунок 1 - Скриншот браузера: страница ЛР7 с мониторингом и управлением устройствами.

Для долговременного хранения реализованы два метода записи в разные коллекции. Метод `insertTemperature` сохраняет документы в коллекцию

`Temperature` с полями времени и значения температуры; при повторении предыдущего значения запись пропускается, чтобы исключить дубли. Метод `insertHeaterSnapshot` сохраняет состояние обогревателя в коллекцию `Heater` при его переключении. Вызовы журналирования выполняются при обновлении температуры и перерасчёте режима обогревателя. На рисунке 2 изображены документы в коллекциях MongoDB.

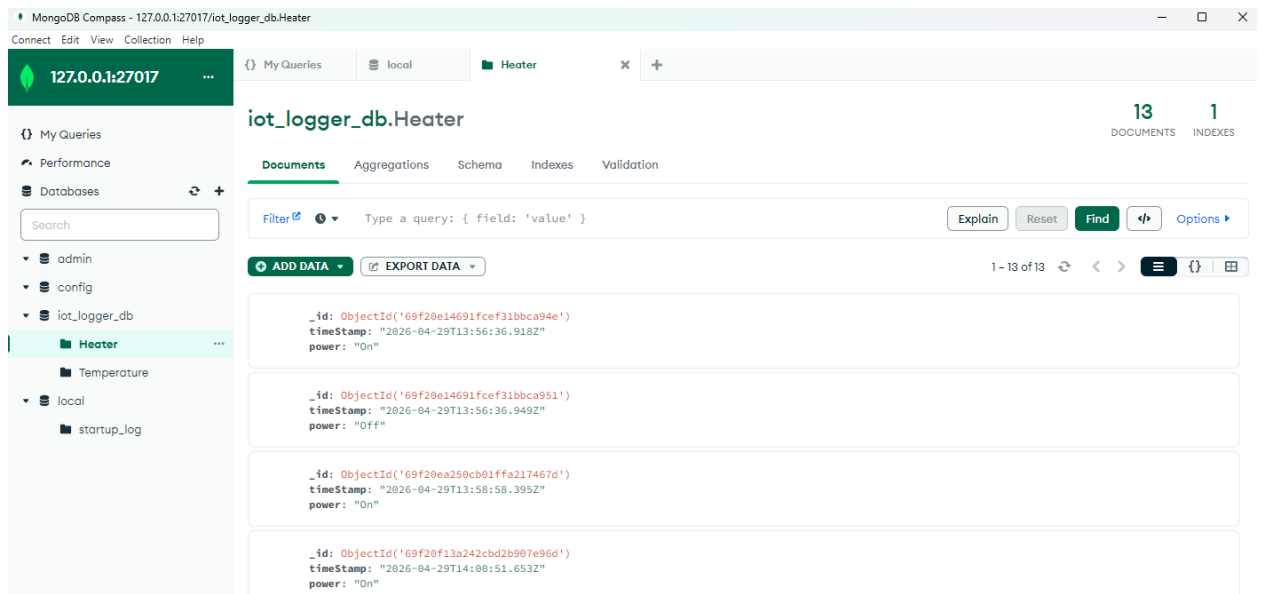


Рисунок 2 - Скриншот MongoDB Compass: коллекции `Temperature` и `Heater` с записями.

В консоли сервера отображаются события подключения к MongoDB, обработка HTTP-запросов и предупреждения при недоступности БД. Это позволяет продемонстрировать отказоустойчивый режим: веб-интерфейс доступен, а логирование временно отключается без остановки приложения. На рисунке 3 изображён фрагмент журнала терминала при запуске и работе системы.

```
C:\WINDOWS\system32\cmd. x + v
[2026-04-29T14:11:19.320Z] GET /connect/light
[Thing.connect] TemperatureSensor (sensor-1)
[HeaterController.autoPower] heater-1: t=11.8°C,apor=25°C -> On
[2026-04-29T14:11:19.321Z] GET /connect/lock
[Thing.connect] LightController (light-1)
[2026-04-29T14:11:19.321Z] GET /connect/heater
[Thing.connect] SmartLock (lock-1)
[Thing.connect] HeaterController (heater-1)
[2026-04-29T14:11:20.314Z] GET /connect/temperature
[2026-04-29T14:11:20.314Z] GET /connect/light
[Thing.connect] TemperatureSensor (sensor-1)
[HeaterController.autoPower] heater-1: t=11.6°C,apor=25°C -> On
[2026-04-29T14:11:20.314Z] GET /connect/lock
[Thing.connect] LightController (light-1)
[2026-04-29T14:11:20.315Z] GET /connect/heater
[Thing.connect] SmartLock (lock-1)
[Thing.connect] HeaterController (heater-1)
[2026-04-29T14:11:21.308Z] GET /connect/temperature
[2026-04-29T14:11:21.308Z] GET /connect/light
[Thing.connect] TemperatureSensor (sensor-1)
[HeaterController.autoPower] heater-1: t=12.1°C,apor=25°C -> On
[2026-04-29T14:11:21.309Z] GET /connect/lock
[Thing.connect] LightController (light-1)
[2026-04-29T14:11:21.310Z] GET /connect/heater
[Thing.connect] SmartLock (lock-1)
[Thing.connect] HeaterController (heater-1)
```

Рисунок 3 - Скриншот терминала: запуск ЛР7 и сообщения логирования/предупреждений.

4 Вывод

В результате выполнения лабораторной работы реализована система долгосрочного хранения данных варианта «Умный дом» с использованием MongoDB и двумя независимыми методами записи в разные коллекции. Полученная реализация обеспечивает накопление телеметрии и событий оборудования при сохранении работоспособности веб-приложения в случае временной недоступности базы данных.

ПРИЛОЖЕНИЕ А

Листинг с кодом программы

В данном разделе представлен исходный код моей части программы:

```
//logger mongodb
import { MongoClient, type Db } from "mongodb";

const DEFAULT_URI = process.env.MONGODB_URI ??
"mongodb://127.0.0.1:27017";

export class IoTLogger {
  private client: MongoClient | null = null;
  private db: Db | null = null;
  private lastLoggedTemperature: number | null = null;
  private enabled = false;

  async init(dbName = "iot_logger_db"): Promise<void> {
    try {
      this.client = new MongoClient(DEFAULT_URI, {
        serverSelectionTimeoutMS: 4000,
        connectTimeoutMS: 4000
      });
      await this.client.connect();
      this.db = this.client.db(dbName);
      this.enabled = true;

      console.log(`[IoTLogger] MongoDB: ${DEFAULT_URI}, БД
«${dbName}»`);
    } catch (err) {
      console.warn("[IoTLogger] подключение к MongoDB не удалось,
логирование отключено:", err);
    }
  }
}
```

```

    this.enabled = false;
    this.db = null;
    if (this.client) {
        try {
            await this.client.close();
        } catch {
            /* ignore */
        }
        this.client = null;
    }
}
}
}

```

```

isEnabled(): boolean {
    return this.enabled && this.db !== null;
}

```

*/** Коллекция Temperature: без дублей подряд одинакового значения (как в пособии). */*

```

async insertTemperature(value: number): Promise<void> {
    if (!this.db || !this.enabled) return;
    if (this.lastLoggedTemperature === value) {
        console.log("[IoTLogger] температура не изменилась — запись не дублируется");
        return;
    }
    this.lastLoggedTemperature = value;
    await this.db.collection("Temperature").insertOne({
        timeStamp: new Date().toISOString(),
        Temperature: value
    });
}

```

```

    });
}

/** Вторая коллекция — снимки состояния обогревателя. */
async insertHeaterSnapshot(power: string): Promise<void> {
    if (!this.db || !this.enabled) return;
    await this.db.collection("Heater").insertOne({
        timeStamp: new Date().toISOString(),
        power
    });
}

async close(): Promise<void> {
    await this.client?.close();
}
}

//обмен данных между датчиком и исполнителем
export class HeaterController extends Thing {
    private power: "On" | "Off" = "Off";

    constructor(id: string, name: string, private readonly switchOnBelowC: number)
    {
        super(id, name, true);
    }

    /** Включение при температуре строго ниже порога (как в пособии, порог
    25 °C). */
    autoPower(temperatureC: number): void {

```

```

    this.power = temperatureC < this.switchOnBelowC ? "On" : "Off";
    console.log(
        `[HeaterController.autoPower] ${this.id}: t=${temperatureC.toFixed(1)}°C,
        порог=${this.switchOnBelowC}°C -> ${this.power}`
    );
}

getPower(): "On" | "Off" {
    return this.power;
}

protected emulate(): void {
    /* состояние задаётся только autoPower, без случайной эмуляции */
}

protected buildPayload(): Record<string, unknown> {
    return {
        id: this.id,
        name: this.name,
        heaterPower: this.power,
        switchOnBelowC: this.switchOnBelowC
    };
}

getStatus(): string {
    return `${this.name}: ${this.power === "On" ? "ВКЛ" : "ВЫКЛ"}`;
}
}

```

// валидация validation.ts

```

function showBad(url) {
  const out = document.getElementById("badDemoOut");
  fetch(url)
    .then(async (r) => {
      const text = await r.text();
      try {
        const j = JSON.parse(text);
        out.textContent = `HTTP ${r.status}\n${JSON.stringify(j, null, 2)}`;
      } catch {
        out.textContent = `HTTP ${r.status}\n${text}`;
      }
    })
    .catch((e) => {
      out.textContent = String(e);
    });
}

```

```

document.getElementById("badTemp").addEventListener("click", () => {
  showBad("/command/temperature?value=abc");
});

```

```

document.getElementById("badLight").addEventListener("click", () => {
  showBad("/command/light?enabled=maybe&brightness=50");
});

```

```

document.getElementById("badLock").addEventListener("click", () => {
  showBad("/command/lock?locked=ckopee_нет");
});

```

```

// app.ts
import express from "express";
import path from "node:path";
import { TemperatureSensor, LightController, SmartLock } from "./things";

const app = express();
const PORT = 5000;

const temp = new TemperatureSensor("sensor-1", "Температурный датчик", 22.5);
const light = new LightController("light-1", "Свет в гостиной");
const lock = new SmartLock("lock-1", "Замок входной двери");

light.setLight(true, 65);
lock.setLocked(true);

app.use((req, _res, next) => {
  if (req.path.startsWith("/connect") || req.path.startsWith("/command")) {
    console.log(`[${new Date().toISOString()}] ${req.method} ${req.path}`);
  }
  next();
});

app.use(express.static(path.join(process.cwd(), "public")));

app.get("/connect/temperature", (_req, res) => res.json(temp.connect()));
app.get("/connect/light", (_req, res) => res.json(light.connect()));
app.get("/connect/lock", (_req, res) => res.json(lock.connect()));

app.get("/command/temperature", (req, res) => {
  const q = req.query as Record<string, string | undefined>;

```

```

const result = temp.applyCommand(q);
if (!result.ok) {
    return res.status(400).json(result);
}
return res.json({ ok: true, state: temp.snapshot() });
});

app.get("/command/light", (req, res) => {
    const q = req.query as Record<string, string | undefined>;
    const result = light.applyCommand(q);
    if (!result.ok) {
        return res.status(400).json(result);
    }
    return res.json({ ok: true, state: light.snapshot() });
});

app.get("/command/lock", (req, res) => {
    const q = req.query as Record<string, string | undefined>;
    const result = lock.applyCommand(q);
    if (!result.ok) {
        return res.status(400).json(result);
    }
    return res.json({ ok: true, state: lock.snapshot() });
});

app.listen(PORT, () => {
    console.log(`http://127.0.0.1:${PORT}`);
});

```

```

//things.ts
export type CommandResult = { ok: true } | { ok: false; error: string };

export abstract class Thing {
  constructor(
    public id: string,
    public name: string,
    public isOnline: boolean = true
  ) {}

  protected abstract emulate(): void
  protected abstract buildPayload(): Record<string, unknown>;

  connect(): Record<string, unknown> {
    console.log(`[Thing.connect] ${this.constructor.name} (${this.id})`);
    this.emulate();
    return this.buildPayload();
  }

  /** Снимок состояния без эмуляции (после команды). */
  snapshot(): Record<string, unknown> {
    return this.buildPayload();
  }

  abstract getStatus(): string;
}

export interface IParent {
  render(statuses: string[]): string;
}

```



```

}

export interface IChild {
  render(statuses: string[]): string;
}

export class MainControlUnit {
  constructor(
    public readonly things: Thing[],
    public readonly datastore: DeviceDataStore
  ) {}

  registerThing(thing: Thing): void {
    console.log(`[MainControlUnit.registerThing] ${thing.id}`);
    this.things.push(thing);
  }

  collectMonitoringData(): void {
    console.log(`[MainControlUnit.collectMonitoringData]`);
    for (const thing of this.things) {
      const line = `[${new Date().toISOString()}] ${thing.getStatus()}`
      this.dataStore.saveRecord(thing, line)
    }
  }

  getAllStatuses(): string[] {
    console.log(`[MainControlUnit.getAllStatuses]`);
    return this.things.map((t) => t.getStatus());
  }
}

export class DeviceDataStore {

```

```

private readonly dataByThingId = new Map<string, string[]>();

saveRecord(thing: Thing, payload: string): void {
  console.log(`[DeviceDataStore.saveRecord] ${thing.id}`);
  const list = this.dataByThingId.get(thing.id) ?? [];
  list.push(payload);
  this.dataByThingId.set(thing.id, list);
}

getRecords(thingId: string): string[] {
  console.log(`[DeviceDataStore.getRecords] ${thingId}`);
  return this.dataByThingId.get(thingId) ?? [];
}

}

export class TemperatureSensor extends Thing {
  private currentTemperatureC: number;

  constructor(id: string, name: string, initialTemperatureC: number) {
    super(id, name, true);
    this.currentTemperatureC = initialTemperatureC;
  }

  setTemperature(valueC: number): void {
    console.log(`[TemperatureSensor.setTemperature] ${this.id}`);
    this.currentTemperatureC = valueC;
  }

  getTemperature(): number {

```

```
    return this.currentTemperatureC;
}
```

```
/** IP4: параметры из query (аналог request.args). */
```

```
applyCommand(query: Record<string, string | undefined>): CommandResult {
    const raw = query.value ?? query.temperature;
    if (raw === undefined || raw === "") {
        return { ok: false, error: "Ожидается параметр value (или temperature)" };
    }
    const n = Number(raw);
    if (Number.isNaN(n)) {
        return { ok: false, error: "value должно быть числом" };
    }
    this.setTemperature(n);
    console.log(`[TemperatureSensor.applyCommand] ${this.id} -> ${n}`);
    return { ok: true };
}
```

```
protected emulate(): void {
    const delta = (Math.random() - 0.5) * 1.2;
    this.currentTemperatureC = Math.round((this.currentTemperatureC + delta) *
10) / 10;
}
```

```
protected buildPayload(): Record<string, unknown> {
    return {
        id: this.id,
        name: this.name,
        value: this.currentTemperatureC,
        unit: "°C"
    }
}
```

```
};  
}
```

```
getStatus(): string {  
    console.log(`[TemperatureSensor.getStatus] ${this.id}`);  
    return `${this.name}: ${this.currentTemperatureC.toFixed(1)} °C`  
}  
}
```

```
export class LightController extends Thing {  
    private brightnessPercent: number = 0;  
    private isEnabled: boolean = false;  
  
    setLight(isEnabled: boolean, brightnessPercent: number): void {  
        console.log(`[LightController.setLight] ${this.id}`);  
        this.isEnabled = isEnabled;  
        this.brightnessPercent = Math.max(0, Math.min(100, brightnessPercent))  
    }  
}
```

```
applyCommand(query: Record<string, string | undefined>): CommandResult {  
    const enabledRaw = query.enabled ?? query.on;  
    const brightRaw = query.brightness ?? query.brightnessPercent;  
    if (enabledRaw === undefined || brightRaw === undefined || brightRaw ===  
    "") {  
        return { ok: false, error: "Ожидаются enabled (true/false) и brightness (0–  
100)" };  
    }  
    const enabled = enabledRaw === "true" || enabledRaw === "1";  
    const brightness = Number(brightRaw);
```

```

    if (Number.isNaN(brightness)) {
        return { ok: false, error: "brightness должно быть числом" };
    }
    this.setLight(enabled, brightness);
    console.log(`[LightController.applyCommand] ${this.id} enabled=${enabled}
brightness=${brightness}`);
    return { ok: true };
}

protected emulate(): void {
    const delta = Math.floor((Math.random() - 0.5) * 12);
    this.brightnessPercent = Math.max(0, Math.min(100, this.brightnessPercent +
delta));
}

protected buildPayload(): Record<string, unknown> {
    return {
        id: this.id,
        name: this.name,
        enabled: this.isEnabled,
        brightnessPercent: this.brightnessPercent
    };
}

getStatus(): string {
    console.log(`[LightController.getStatus] ${this.id}`);
    return `${this.name}: ${this.isEnabled ? "ВКЛ" : "ВЫКЛ"},
яркость=${this.brightnessPercent}%`;
}
}

```

```

export class SmartLock extends Thing{
  private isLocked: boolean = true;

  setLocked(value: boolean): void {
    console.log(`[SmartLock.setLocked] ${this.id}`);
    this.isLocked = value;
  }

  applyCommand(query: Record<string, string | undefined>): CommandResult {
    const raw = query.locked ?? query.lock;
    if (raw === undefined || raw === "") {
      return { ok: false, error: "Ожидается locked (true/false)" };
    }
    if (raw !== "true" && raw !== "false" && raw !== "1" && raw !== "0") {
      return { ok: false, error: "locked должно быть true или false" };
    }
    const locked = raw === "true" || raw === "1";
    this.setLocked(locked);
    console.log(`[SmartLock.applyCommand] ${this.id} locked=${locked}`);
    return { ok: true };
  }

  protected emulate(): void {
    if (Math.random() < 0.08) this.isLocked = !this.isLocked;
  }

  protected buildPayload(): Record<string, unknown> {
    return {
      id: this.id,

```

```

        name: this.name,
        locked: this.isLocked
    };
}

getStatus(): string {
    console.log(`SmartLock.getStatus ${this.id}`);
    return `${this.name}: ${this.isLocked ? "ЗАКРЫТ" : "ОТКРЫТ"}`;
}
}

export class ParentInterface implements IParent {
    render(statuses: string[]): string {
        console.log("[ParentInterface.render]");
        return ["Интерфейс родителя (полный доступ)", ...statuses].join("\n");
    }
}

export class ChildInterface implements IChild {
    render(statuses: string[]): string {
        console.log("[ChildInterface.render]");
        return ["Интерфейс ребенка (ограничен)", ...statuses].join("\n");
    }
}

export function runDemo(): {
    parentText: string;
    childText: string;
    sensorRecords: string[];
} {

```

```

const store = new DeviceDataStore();
const things: Thing[] = [];
const mcu = new MainControlUnit(things, store);
  const temp = new TemperatureSensor("sensor-1", "Температурный датчик",
22.5);
  const light = new LightController("light-1", "Свет в гостиной");
  const lock = new SmartLock("lock-1", "Замок входной двери");
  light.setLight(true, 65);
  lock.setLocked(true);
  mcu.registerThing(temp);
  mcu.registerThing(light);
  mcu.registerThing(lock);
  mcu.collectMonitoringData();
  const statuses = mcu.getAllStatuses();
  const parentText = new ParentInterface().render(statuses);
  const childText = new ChildInterface().render(statuses);
  return {
    parentText,
    childText,
    sensorRecords: store.getRecords("sensor-1")
  };
}

```

//script.js

```

const CONNECT = {
  temperature: "/connect/temperature",
  light: "/connect/light",
  lock: "/connect/lock",
};

```



```

async function fetchJson(url) {
  const response = await fetch(url);
  if (!response.ok) {
    throw new Error(`HTTP ${response.status} для ${url}`);
  }
  return response.json();
}

//index.js
function $(id) {
  return document.getElementById(id);
}

async function updateTemperature() {
  const data = await fetchJson(CONNECT.temperature)
  document.getElementById("tempValue").textContent = `${data.value}
  ${data.unit}`
}

async function updateLight() {
  const data = await fetchJson(CONNECT.light);
  $("lightEnabled").textContent = data.enabled ? "ВКЛ" : "ВЫКЛ";
  $("lightBrightness").textContent = String(data.brightnessPercent);
}

async function updateLock() {
  const data = await fetchJson(CONNECT.lock);
  $("lockState").textContent = data.locked ? "ЗАКРЫТ" : "ОТКРЫТ";
}

async function tick() {

```

```
    await Promise.all([updateTemperature(), updateLight(), updateLock()]);
  }
```

```
tick();
```

```
setInterval(tick, 1000);
```

```
//index.html
```

```
<!DOCTYPE html>
```

```
<html lang="ru">
```

```
<head>
```

```
  <meta charset="UTF-8" />
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

```
  <title>IoT Дом — ЛР4</title>
```

```
  <style>
```

```
    body { font-family: system-ui, sans-serif; max-width: 42rem; margin: 1.5rem
auto; padding: 0 1rem; }
```

```
    section { border: 1px solid #ddd; border-radius: 8px; padding: 0.75rem 1rem;
margin: 1rem 0; }
```

```
    h1 { font-size: 1.35rem; }
```

```
    code { background: #f4f4f4; padding: 0.1rem 0.35rem; border-radius: 4px; }
```

```
    .result { font-size: 0.9rem; color: #333; margin-top: 0.35rem; }
```

```
  </style>
```

```
</head>
```

```
<body>
```

```
  <h1>ЛР4 — мониторинг (ЛР3) + команды управления</h1>
```

```
  <section>
```

```
    <h2>Мониторинг</h2>
```

```
    <p>Обновление раз в секунду через <code>GET /connect/...</code></p>
```

```
    <p><strong>Температура:</strong> <span id="tempValue">—</span></p>
```

```
<p><strong>Свет:</strong> <span id="lightEnabled">—</span>, яркость
<span id="lightBrightness">—</span>%</p>
```

```
<p><strong>Замок:</strong> <span id="lockState">—</span></p>
</section>
```

```
<section>
```

```
<h2>Управление</h2>
```

```
<p>Отправка на <code>GET /command/...</code> с параметрами в строке
запроса.</p>
```

```
<div>
```

```
<h3>Температура</h3>
```

```
<input id="cmdTemp" type="text" placeholder="например 23.5" />
```

```
<button id="btnTemp" type="button">Отправить</button>
```

```
<div class="result" id="cmdTempResult"></div>
```

```
</div>
```

```
<div>
```

```
<h3>Свет</h3>
```

```
<label><input id="cmdLightOn" type="checkbox" checked />
Включено</label>
```

```
<input id="cmdLightBright" type="number" min="0" max="100" value="65"
/>
```

```
<button id="btnLight" type="button">Отправить</button>
```

```
<div class="result" id="cmdLightResult"></div>
```

```
</div>
```

```
<div>
```

```
<h3>Замок</h3>
```

```

        <label><input id="cmdLockLocked" type="checkbox" checked />
Закрѳт</label>
        <button id="btnLock" type="button">Отправить</button>
        <div class="result" id="cmdLockResult"></div>
    </div>
</section>

<script src="/script.js"></script>
<script src="/index.js"></script>
<script src="/command.js"></script>
</body>
</html>

```

```
//command.js
```

```

async function sendCommand(url) {
    const response = await fetch(url);
    const data = await response.json();
    if (!response.ok) {
        throw new Error(JSON.stringify(data));
    }
    return data;
}

```

```

function setResult(id, text) {
    const el = document.getElementById(id);
    if (el) el.textContent = text;
}

```

```

document.getElementById("btnTemp").addEventListener("click", async () => {
    const value = document.getElementById("cmdTemp").value.trim();

```

```

try {
    const data = await sendCommand(
        `/command/temperature?value=${encodeURIComponent(value)}`
    );
    setResult("cmdTempResult", JSON.stringify(data));
} catch (e) {
    setResult("cmdTempResult", String(e.message));
}
});

document.getElementById("btnLight").addEventListener("click", async () => {
    const enabled = document.getElementById("cmdLightOn").checked;
    const brightness = document.getElementById("cmdLightBright").value;
    const qs = new URLSearchParams({
        enabled: String(enabled),
        brightness: String(brightness)
    });
    try {
        const data = await sendCommand(`/command/light?${qs.toString()}`);
        setResult("cmdLightResult", JSON.stringify(data));
    } catch (e) {
        setResult("cmdLightResult", String(e.message));
    }
});

```

```

document.getElementById("btnLock").addEventListener("click", async () => {
    const locked = document.getElementById("cmdLockLocked").checked;
    const qs = new URLSearchParams({ locked: String(locked) });
    try {
        const data = await sendCommand(`/command/lock?${qs.toString()}`);

```

```
        setResult("cmdLockResult", JSON.stringify(data));  
    } catch (e) {  
        setResult("cmdLockResult", String(e.message));  
    }  
});
```